Operating Systems

Abdelrahman Hosny

2026

Faculty of Computers and Information, Assiut University

Contents

01
File Systems

02
Processes

03
Threads

04
Virtual Memory

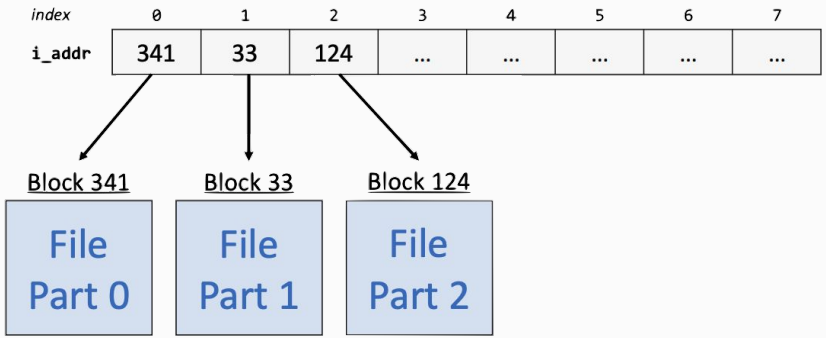
05
Miscellaneous

06
The Shell

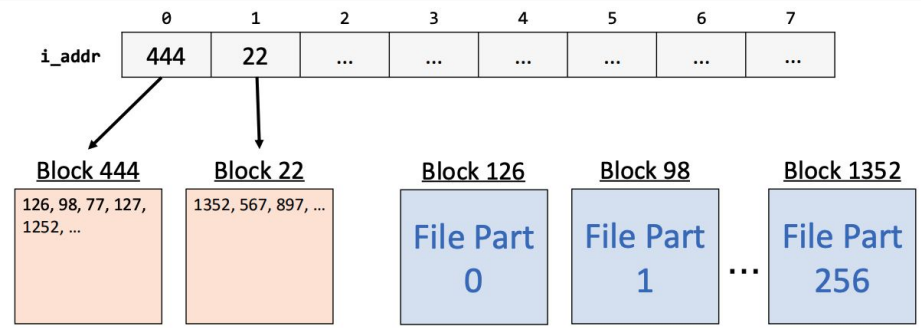
Announcements

- **No class** on Thursday March 19th, 2026 – Eid Mubarak!
- **No labs** this week – Ramadan kareem!
- **Project 1** will be released Sunday March 22nd, 2026.
    - Deliver by: Friday April 10th, 2026.
- **Midterm** will likely be in the week of April 5th or 12th

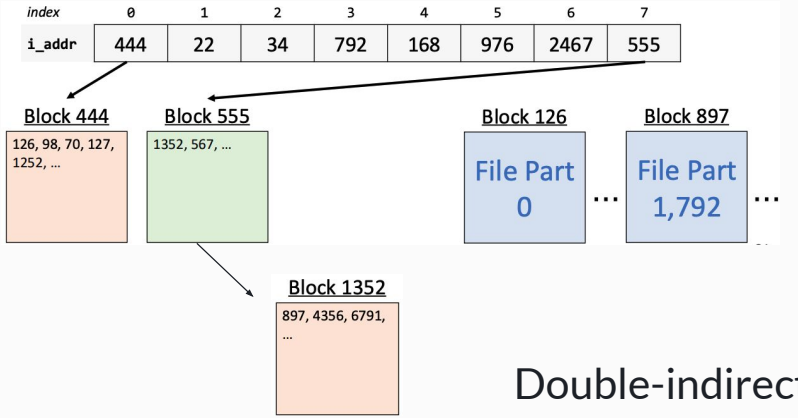
Summary - Non-contiguous Allocation



Direct addressing: 4KB



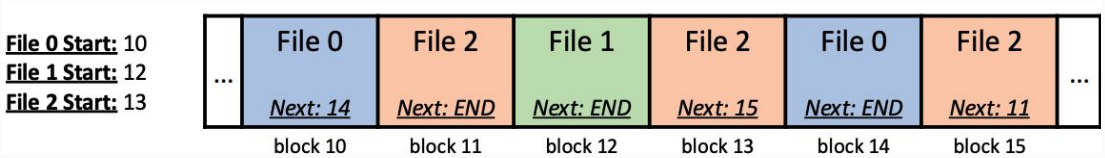
Single-indirect addressing: 1MB



Double-indirect addressing

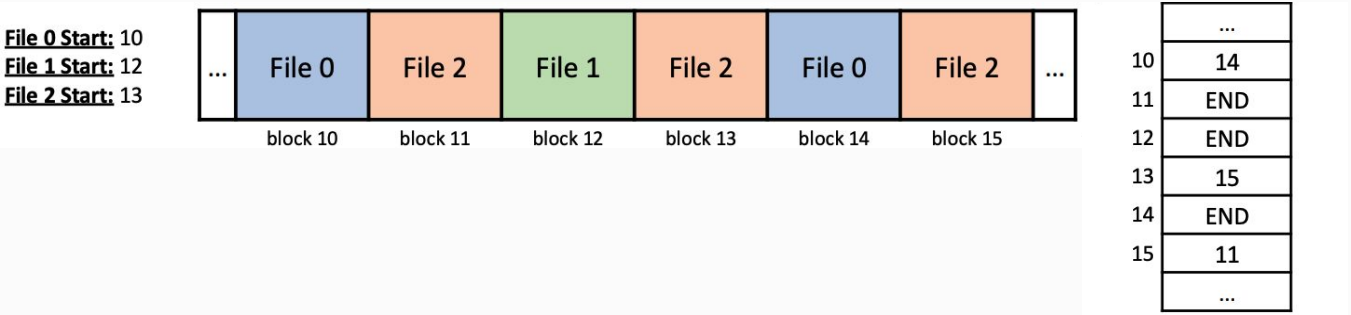
Summary - inodes

Files as linked lists

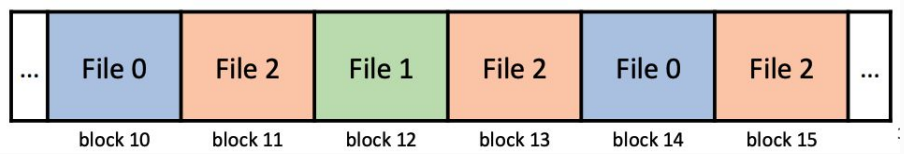


In-memory

File Allocation Table



Multi-level Index



File 0: 10, 14

File 1: 12

File 2: 13, 15, 11

Summary - lab 2

```
This message is shown once a day. To disable it please create the
/home/mohamed_g/.hushlogin file.
mohamed_g@DESKTOP-O3116FE:~$ mkdir -p ~/fs_lab && cd ~/fs_lab
mohamed_g@DESKTOP-O3116FE:~/fs_lab$ echo "Hello Filesystem" > small.txt
mohamed_g@DESKTOP-O3116FE:~/fs_lab$ ls -li small.txt
60798594969648047 -rw-r--r-- 1 mohamed_g mohamed_g 17 Mar 10 10:45 small.txt
mohamed_g@DESKTOP-O3116FE:~/fs_lab$ stat small.txt
  File: small.txt
  Size: 17              Blocks: 0          IO Block: 512    regular file
Device: 0,2     Inode: 60798594969648047  Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/mohamed_g)   Gid: ( 1000/mohamed_g)
Access: 2026-03-10 10:45:25.241052900 +0200
Modify: 2026-03-10 10:45:25.241052900 +0200
Change: 2026-03-10 10:45:25.241052900 +0200
 Birth: -
```

What could be a reason that `Blocks: 0` ?

What is the difference between Blocks and IO Block?

This Lecture

Directories & Lookups

Directories

Filesystems usually support directories ("folders")

- A directory can contain files and more directories
- A directory is a file container. It needs to store information about what files/folders are contained within it.

Directories

A Unix V6 directory's payload data is an unsorted list of 16 byte "directory entries".

Each entry contains the name and inode number of one thing in that directory.

- The first two bytes are the inumber
- The last 14 bytes are the name (not necessarily null-terminated!)

```
struct direntv6 {
  uint16_t d_inumber;
  char     d_name[14];
};
```

23	myfile.txt
54	song.mp3
1245	prez.pptx
...	

Directories

How can we store directories on disk?

- Directories have payload data: directory entries (could be many entries)
- Directories have metadata (size, permissions, creation date, …)

Directories

Let's model a directory as a file. We'll pretend it's a "file" whose contents are its directory entries!

→ So each directory will have an inode

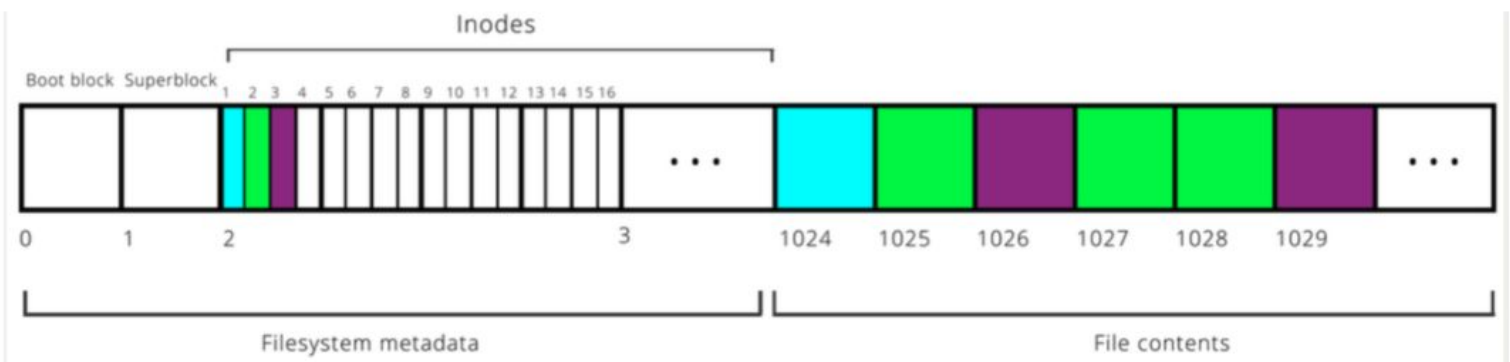
we can leverage all the existing logic for how files and inodes work, no need for extra work or complexity!

- Inodes can store a field telling us whether something is a directory or file.
- Directories can be "small mode" or "large mode", just like files

Directories .. but ?

We understand how files and folders are stored.

But how do we find them?



We need a way to look up!

The directory hierarchy

On Unix/Linux, all files live within the root directory, "/"

- We can specify the location of a file via the path to it from the root directory (“absolute path”):

  `/classes/operating-systems/hello.txt`

- Common filesystem task: given a filepath, get the file's contents.

The lookup process

`/classes/operating-systems/hello.txt`

The root directory ("/") is set to have inumber 1. That way we always know where to go to start traversing. (0 is reserved to mean "NULL" or "no inode")

The lookup process

`/classes/operating-systems/hello.txt`

Start at the root directory

Go to inode with inumber 1

(root directory).

The lookup process

`/classes/operating-systems/hello.txt`

In the root directory, find the entry named "classes".

In its payload data, look for the entry “classes” and get its inumber. Go to that inode.

The lookup process

`/classes/operating-systems/hello.txt`

In the "classes" directory, find the entry named "operating-systems"

In its payload data, look for the entry “operating-systems” and get its inumber. Go to that inode.

The lookup process

`/classes/operating-systems/hello.txt`

In the "operating-systems" directory, find the entry named "hello.txt". Then read its contents.

In its payload data, look for the entry “hello.txt” and get its inumber. Go to that inode and read in its payload data.

Practice #1

What is the inode number for the file with path /local/files/story.txt?

Inode #	1	…	3	…	12	…	14	…	16
Inode contents	Type: dir Mode: small Size: 80 i_addr = [24, …]	…	Type: file Mode: small Size: 1536 i_addr = [128, 222, 124, …]	…	Type: dir Mode: small Size: 544 i_addr = [32, 41, …]	…	Type: dir Mode: small Size: 64 i_addr = [62, …]	…	Type: file Mode: large Size: 4608 i_addr = [125, …]

Block #	…	2	…	24	…	32	…	41	…	62	…	128	…
Block contents	…	*Start of inode table*	…	. 1 .. 1 local 12 other 10 remote 9	…	. 12 .. 1 file1.txt 4 docs 15 ... ("files" not here)	…	apps 21 files 14	…	. 14 .. 12 story.txt 3 todo.txt 16	…	Once upon a time…	…

Practice #2

What is the inode number for the file with path /usr/note.txt?

Inode #	1	…	3	…	12	…	14	…	16
Inode contents	Type: dir Mode: small Size: 96 i_addr = [56, …]	…	Type: dir Mode: large Size: 131584 i_addr = [122, 545, …]	…	Type: dir Mode: small Size: 544 i_addr = [32, 41, …]	…	Type: dir Mode: small Size: 64 i_addr = [62, …]	…	Type: file Mode: large Size: 4608 i_addr = [876, …]

Block #	…	2	…	56	…	67	…	122	…	421	…	545	…
Block contents	…	*Start of inode table*	…	. 1 .. 1 bin 13 tmp 10 other 9 usr 3	…	. 3 .. 1 apps 21 files 14 … ("note.txt" not here)	…	67,421,872, 999,135,346 ,…	…	icon.png 30 doc.pdf 15 note.txt 16 …	…	565	…

Unix v6 Filesystem Summary

Every file has an inode (stores block numbers and other metadata)

- “small” (up to 8 direct) or “large” (up to 7 singly-indirect and 1 doubly-indirect)
- Every directory also has an inode. Directories store directory entries.
- Directory entry = inumber + name
- We leverage directories to look up by name
- For looking up by absolute path, we start at root directory (inumber 1)

Unix v6 Filesystem Summary

The Unix V6 Filesystem is one example of a “multi-level index” filesystem design.

Advantages

- Can access all block numbers for a file
- Still supports easy sequential access
- Easy to grow files

Unix v6 Filesystem Summary

The Unix V6 Filesystem is one example of a “multi-level index” filesystem design.

Disadvantages

- More steps and disk reads to get block data for large files
- More disk space taken up by metadata
- Upper limit on file size (though if larger than disk, doesn’t matter)
- Size change requires restructuring the inode

Other ideas

There are many alternative designs that could be used – For example:

- What if the block size was different?
- What if inodes stored a different number of block numbers?
- What if the file size scheme (small / large) worked differently?

Example: 4.3 BSD Unix filesystem (evolutionary descendent of V6)

- 4Kb block size
- Inodes store 14 block numbers
- First 12 block numbers always direct, 13th always singly indirect, 14th always doubly indirect (no small vs. large schemes)

Other ideas

Larger block size? Improves efficiency of I/O and inodes but worsens internal fragmentation.

Generally: challenges with both large and small files coexisting.

One idea: multiple block sizes

- Large blocks are 4KB, fragments are 512 bytes (8 fragments fit in a block)
- The last block in a file can be 0-7 fragments
- One large block can hold fragments from multiple files
- Get the time efficiency benefit of larger blocks, but the internal fragmentation benefit of smaller blocks (small files can use fragments)

Other ideas

Filesystem design is a hard problem! Tradeoffs, challenges with large and small files.

- Reallocate files as blocks grow – initially allocate blocks one at a time, but when a file reaches a certain size, reallocate blocks looking for large contiguous clusters

- ext4 is a popular current Linux filesystem – you may notice similarities!
- NTFS (replacement for FAT) is the current Windows filesystem
- APFS (“Apple Filesystem”) is the filesystem for Apple devices

Take home exercise

What do folders/files under “/” do?

Any questions?
No stupid question!

Let's discuss any doubts or concerns.
If something comes up later, contact abdelrahman@aun.edu.eg